

Processor-Level Parallelism

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory
Department of Electrical and Computer Engineering
Cullen College of Engineering
University of Houston

UNIVERSITY of **HOUSTON** | ECE

Serial vs. Parallel Computing

- Serial Programming:
 1. Formulate the problem as a series of instructions
 2. Each instruction is executed
 3. Only one instruction is executed at a time
- Parallel Programming attempts to execute instructions or parts of instructions simultaneously
- Types of parallelism:
 - **Bit** level – achieved by increasing processor word size **compiler + processor**
 - **Instruction** level – execute independent instructions simultaneously **processor**
 - **Task** level – complex tasks executed by independent processors **programmer**
 - **Data** level – similar tasks executed independently on different data **programmer**

Bit-Level Parallelism

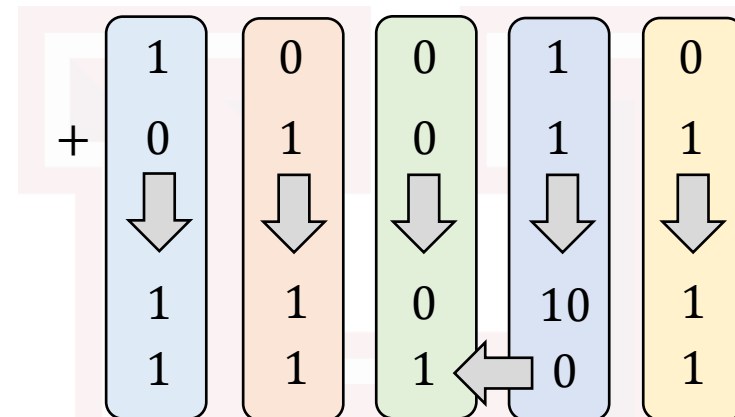
- word – size of a processor register
 - stores a single operand for an operation
 - modern desktops use 64-bit operands
- Perform bit-wise operations in parallel
 - (word size must be known)

calculate $\pi - \sqrt{2}$ using 32-bit floating point:

	exponent	mantissa
$\pi = 0$	10000000	10010010000111111011011
$-\sqrt{2} = 1$	01111111	01101010000010011110011

shift smaller mantissa:

1.01101010000010011110011
0.101101010000010011110011



difference between exponents: 10000000_2
 -01111111_2

 00000001_2



add mantissas: $1.100100100001111110110110_2$
 $-0.101101010000010011110011_2$

 $0.110111010001101011000011_2$



Instruction-Level Parallelism

- What is a program?

$$a = x^2 + y^2 + z^2$$

compile

```
float a = x*x + y*y + z*z;
```

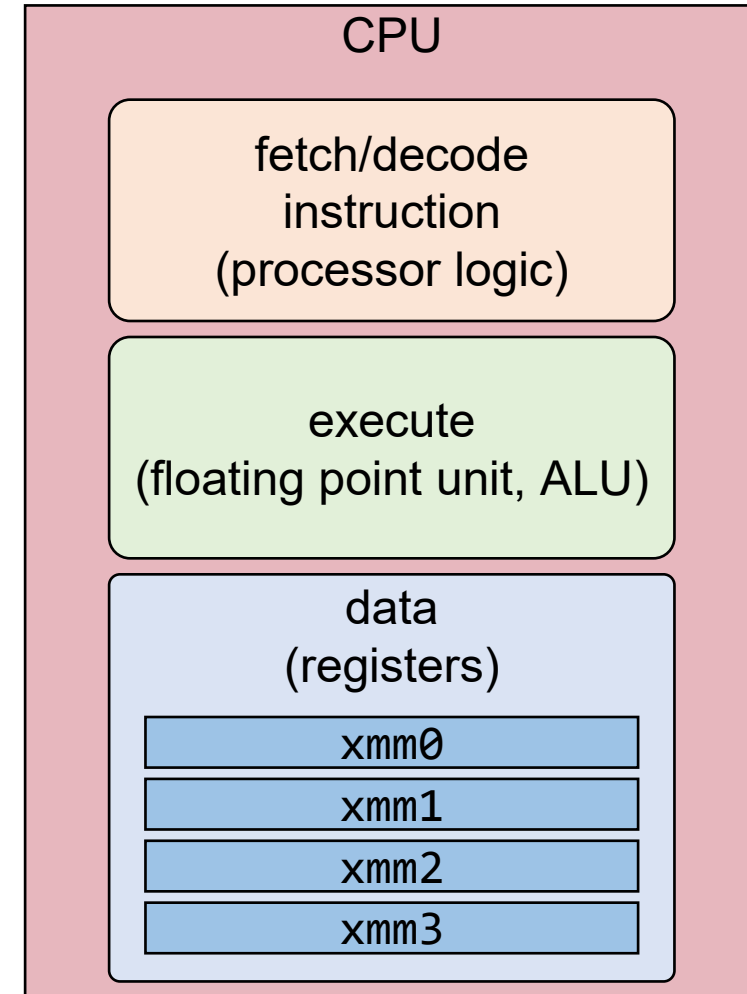
```
float a = x*x + y*y + z*z;
```

- Set of operations
- Performed on data

```
mov    xmm1, DWORD PTR x[rip]
mov    xmm0, DWORD PTR x[rip]
mul     xmm1, xmm0
mov     xmm2, DWORD PTR y[rip]
mov     xmm0, DWORD PTR y[rip]
mul     xmm0, xmm2
add     xmm1, xmm0
mov     xmm2, DWORD PTR z[rip]
mov     xmm0, DWORD PTR z[rip]
mul     xmm0, xmm2
add     xmm0, xmm1
mov     DWORD PTR a[rip], xmm0
```

Basic Processors (from a programming POV)

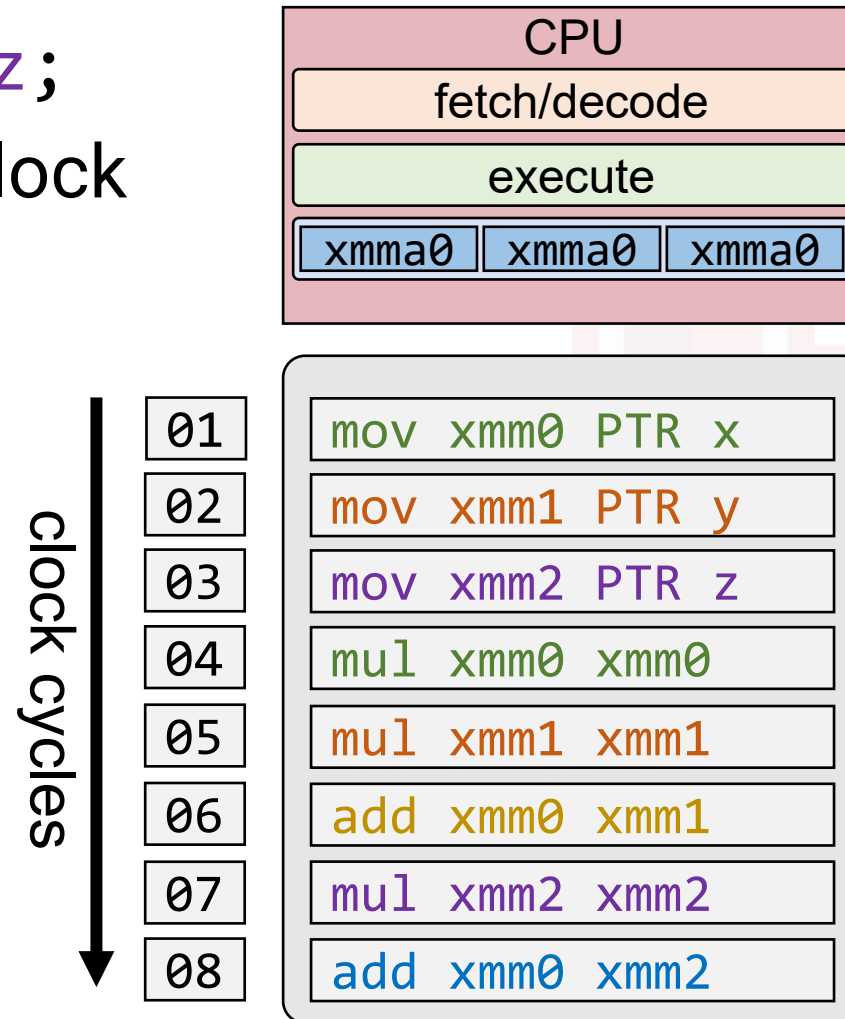
- What is a processor?
- Performs operations:
 1. determine what operation to perform (fetch/decode)
 2. retrieve data from memory
 3. execute the operation
 - move values to registers
 - perform arithmetic on registers
 4. move register data into memory



Process Execution

```
float a = x*x + y*y + z*z;
```

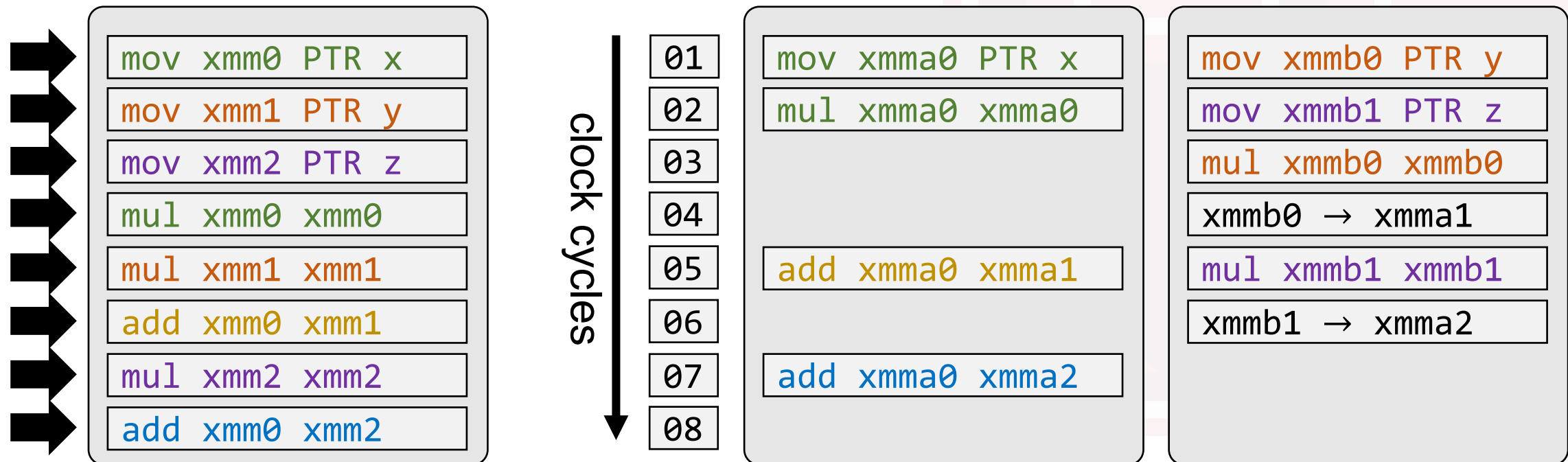
- Assume one execution per clock



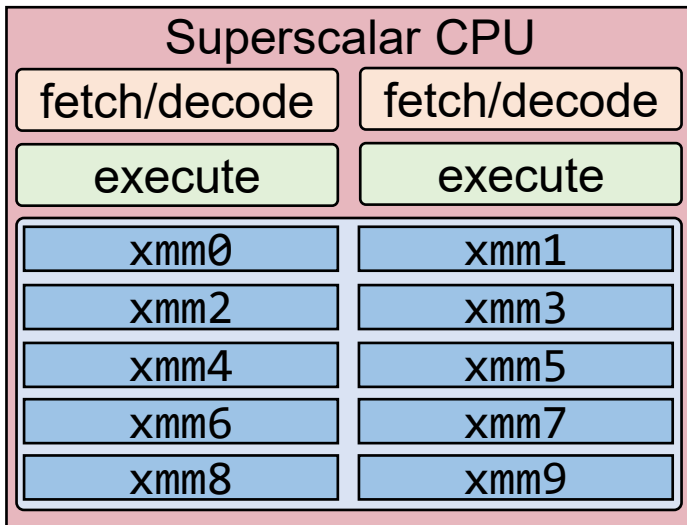
Process Execution (2 Processors)

float **a** = **x*****x** + **y*****y** + **z*****z**;

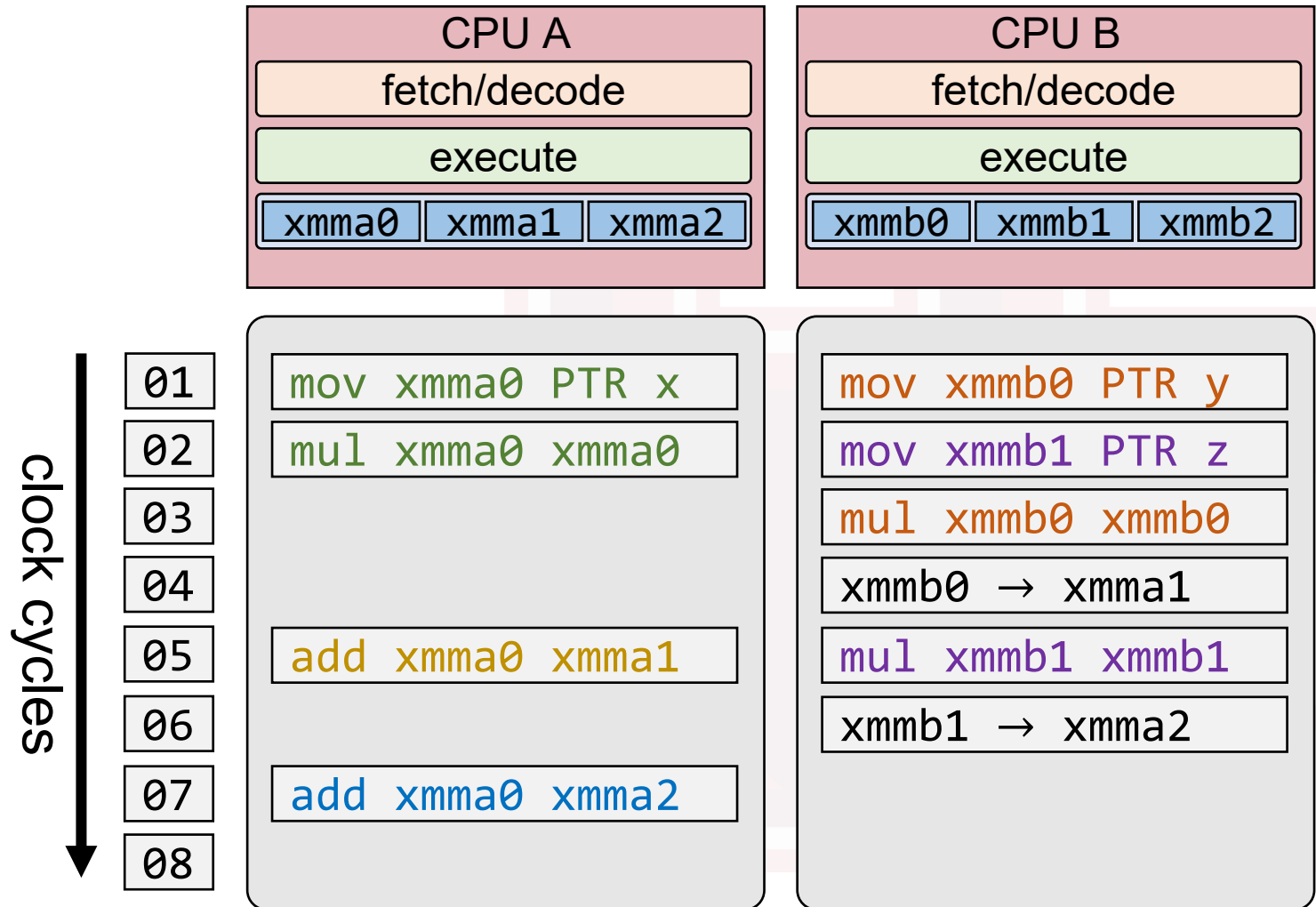
- Assume one execution per clock



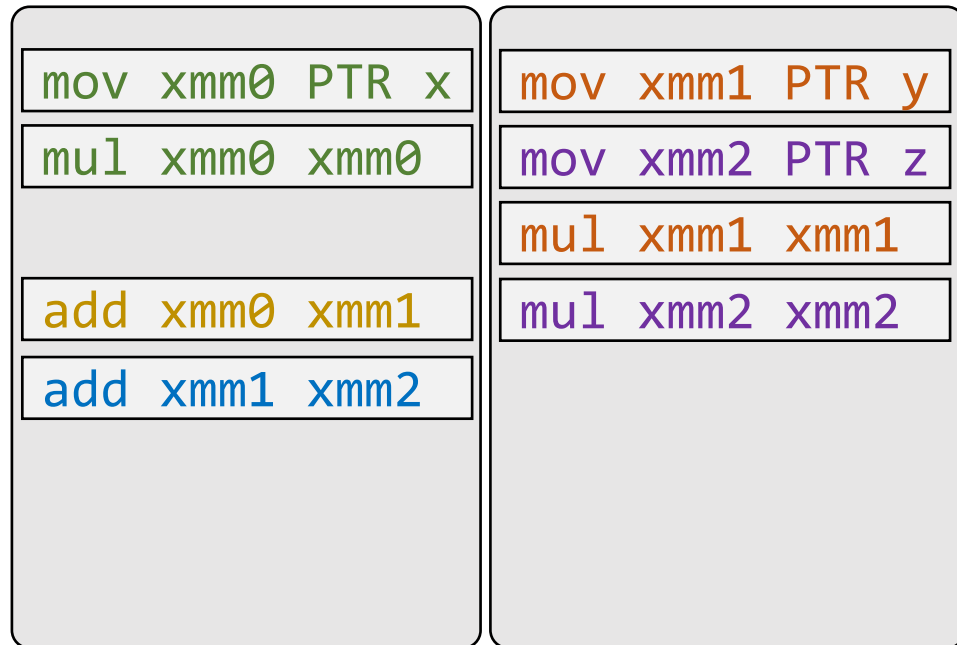
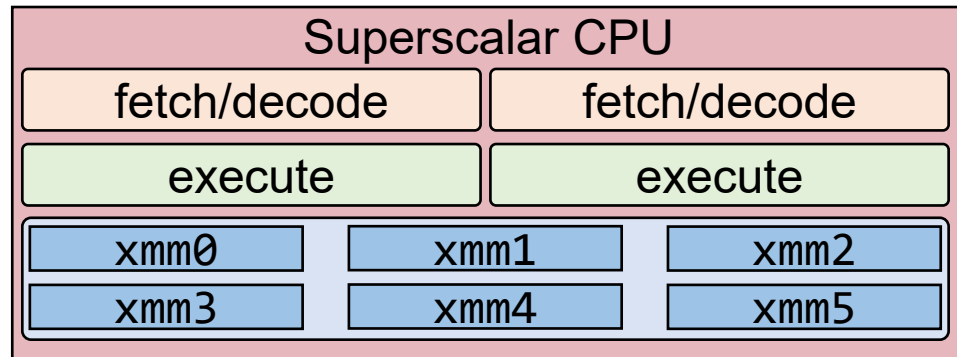
Superscalar Processors



- Identifies dependencies between instructions
- Executes independent instructions in parallel

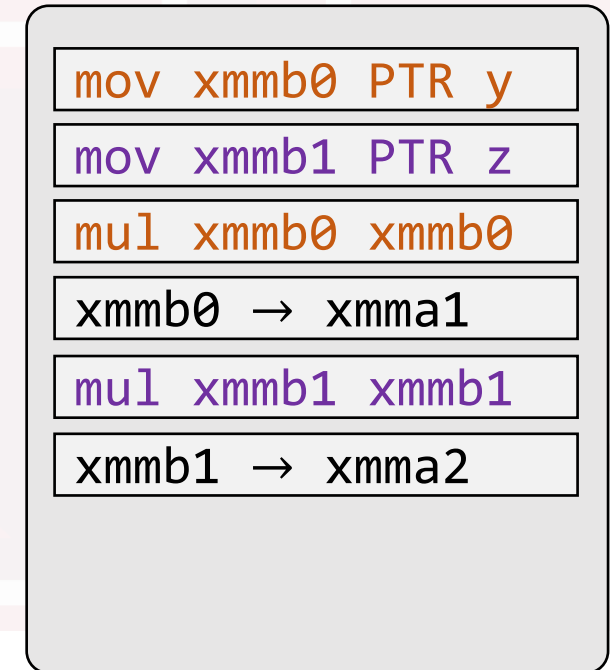
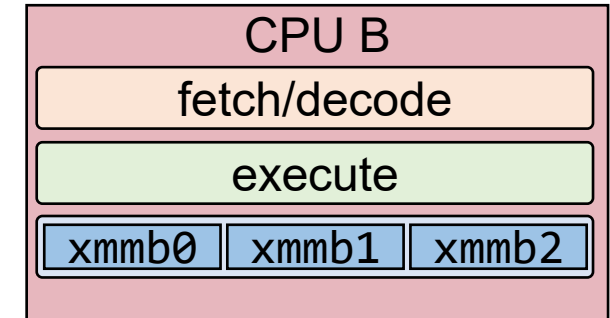
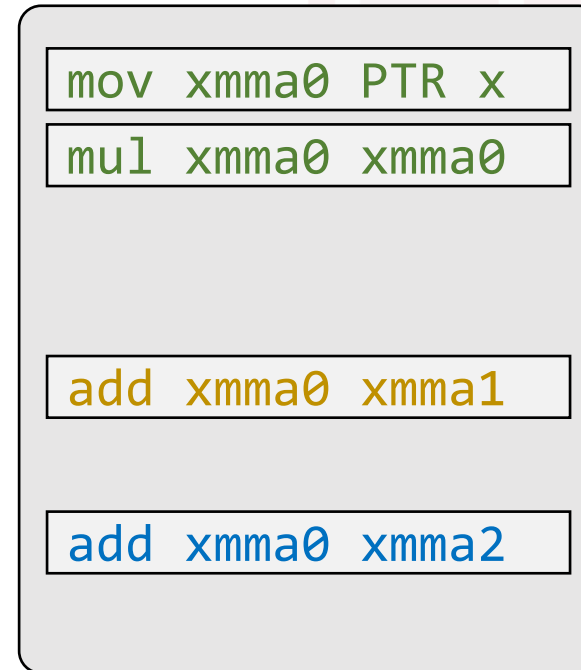
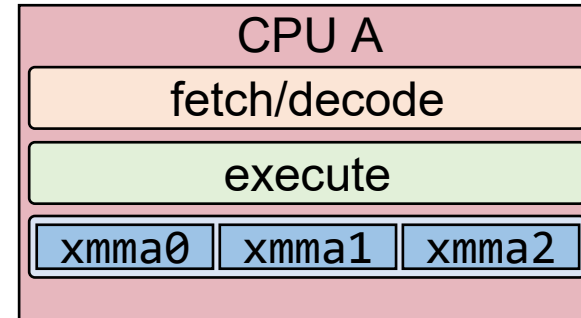


Superscalar Processors



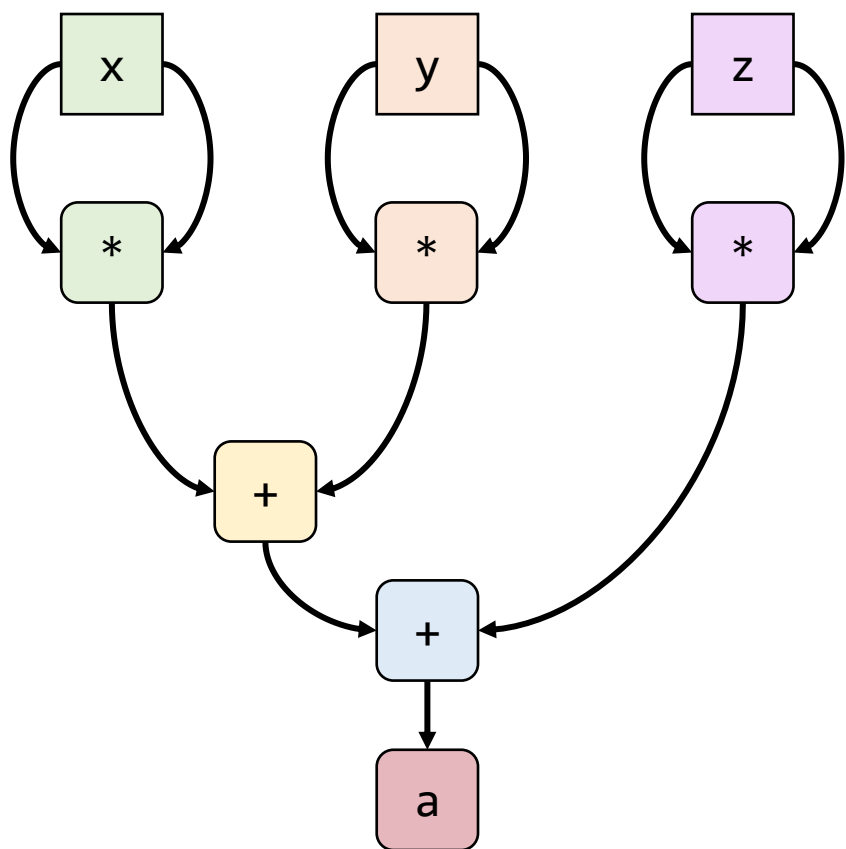
clock cycles

01
02
03
04
05
06
07
08

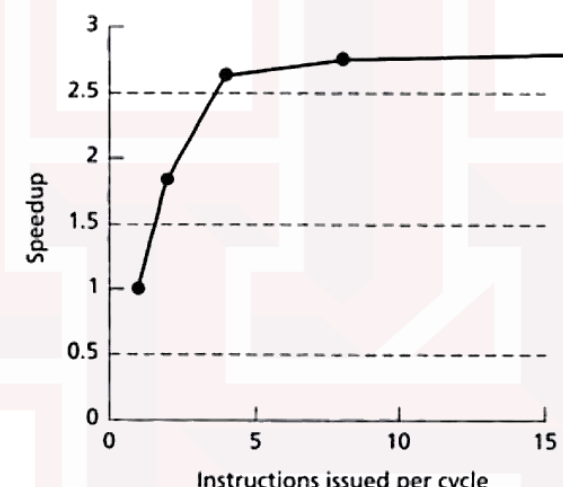
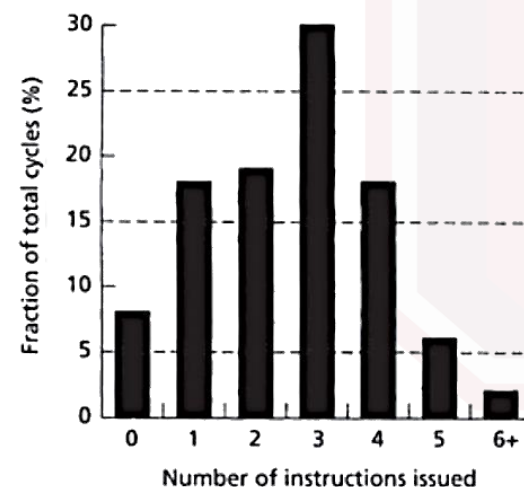


Dependencies Limit Superscalar Performance

float **a** = **x*****x** + **y*****y** + **z*****z**;



- Modern CPUs are superscalar
- Limited instruction-level parallelism



Culler, Singh, and Gupta (1998)

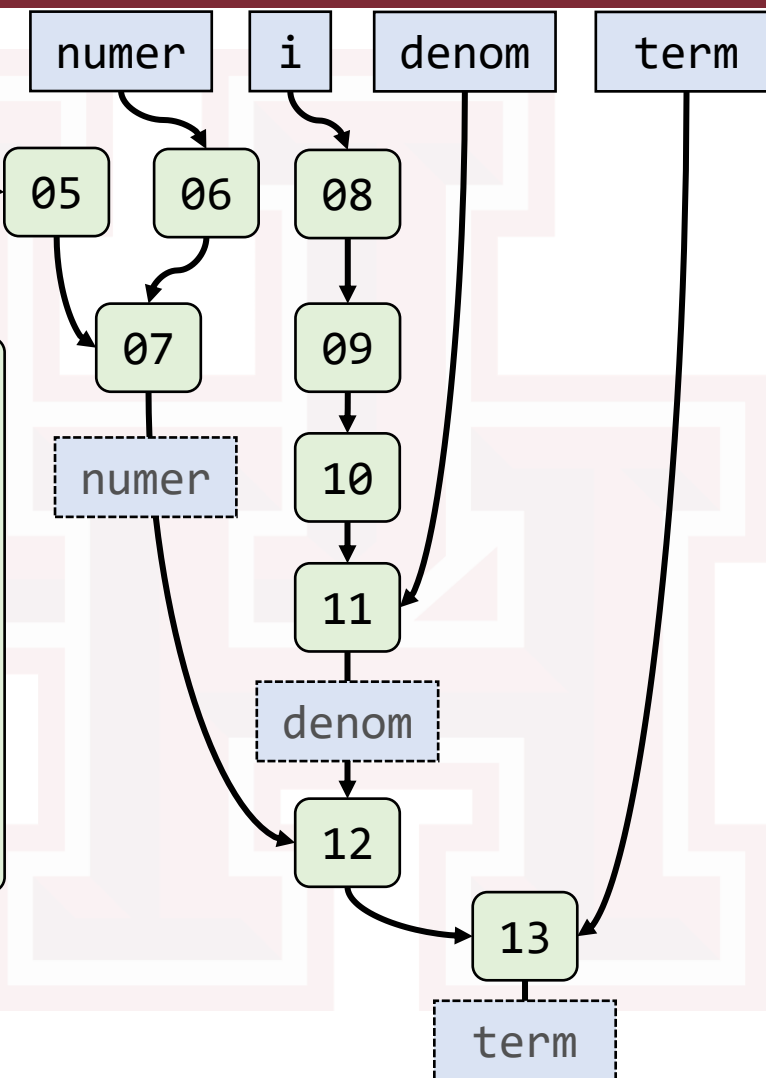
- Additional logic takes up valuable chip space

Dependency Graphs

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{1+2n}}{(1+2n)!}$$

```
float sine(float x, int n) {
    float number = x;
    float denom = 1;
    float term = x;
    for(int i = 1; i <= n; i++) {
        number = number * (-1) * x * x;
        denom = denom * (2 * i) * (2 * i + 1);
        term = term + (number / denom);
    }
}
```

```
float comp_sine(float x, int n) {
01  float number = x;
02  float denom = 1;
03  float term = x;
04  for(int i = 1; i <= n; i++) {
05      float tmp0 = x * x;
06      float tmp1 = number * (-1);
07      number = tmp0 + tmp1;
08      float tmp2 = 2 * i;
09      float tmp3 = tmp2 + 1;
10      float tmp4 = tmp2 * tmp3;
11      denom = denom * tmp4;
12      float tmp5 = number / denom;
13      term = term + tmp5;
    }
}
```



Moore's Law: The number of transistors on microchips has

- Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing power, memory capacity, and energy efficiency.

Moore's Law

- Transistor counts double every one and a half years



OurWorldinData.org – Research and data to make progress against the world's largest problems.

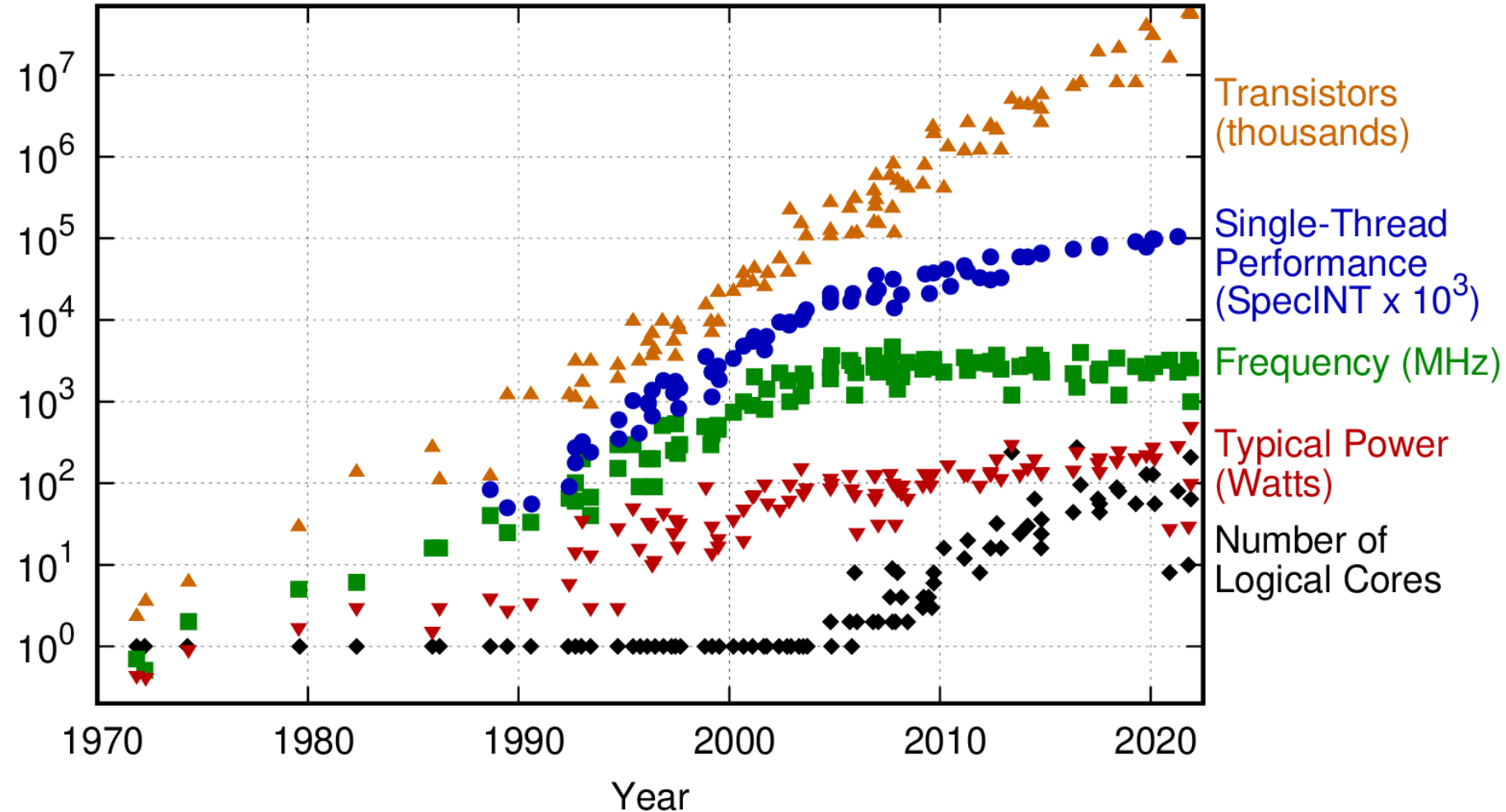
Licensed under [CC-BY](#) by the authors Hannah Ritchie and Max Roser.

Limitations to Single-Process Scaling

- **Clock rate:** the frequency at which a processor can execute instructions
 - synchronize operations between components on a microchip
 - generally limited by the distance between components and heat
- **Transistor fabrication size:** dimensions of transistors that can be created
 - generally limited by the feature size achievable using photolithography
 - also limited by quantum effects (ex. tunneling) that introduce errors/noise
- (lack of) **Dennard scaling:** power requirements reduce with MOSFET size
 - until around 2006, shrinking a transistor also reduced its power requirements
 - threshold voltage and leakage current don't scale as well (now chips get hotter as they get smaller)

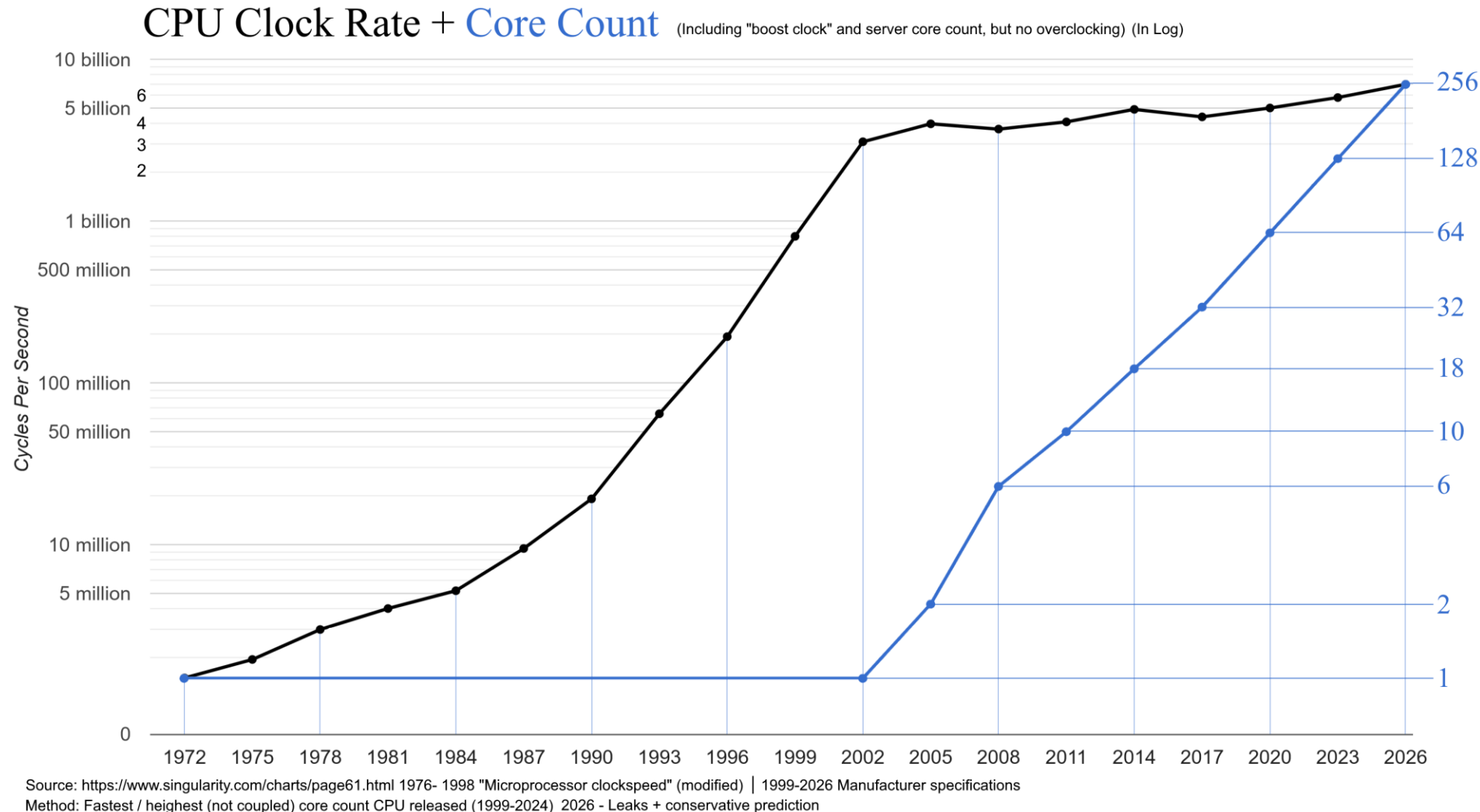
General Trends

<https://github.com/karlrupp/microprocessor-trend-data>

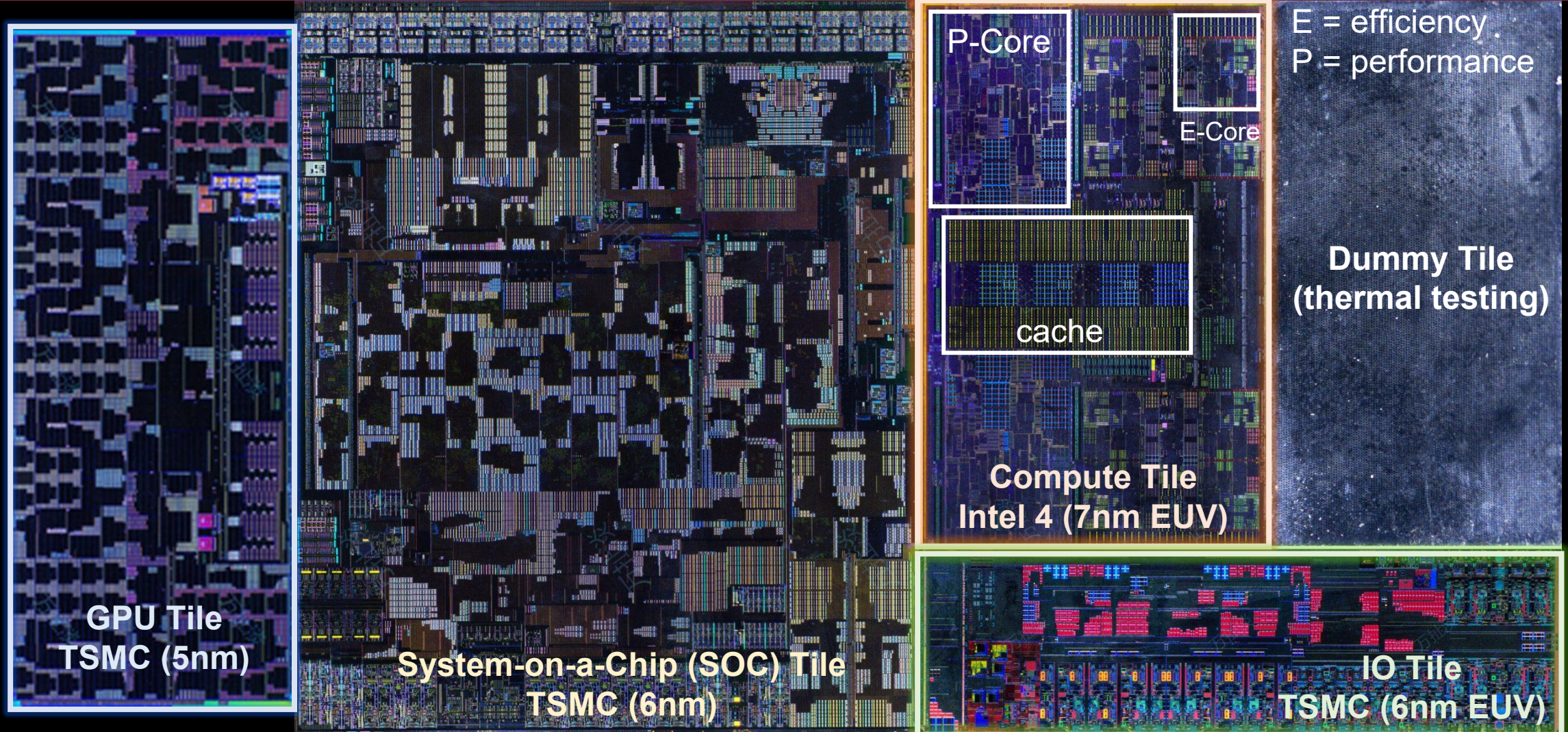


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

Synchronizing Across a CPU

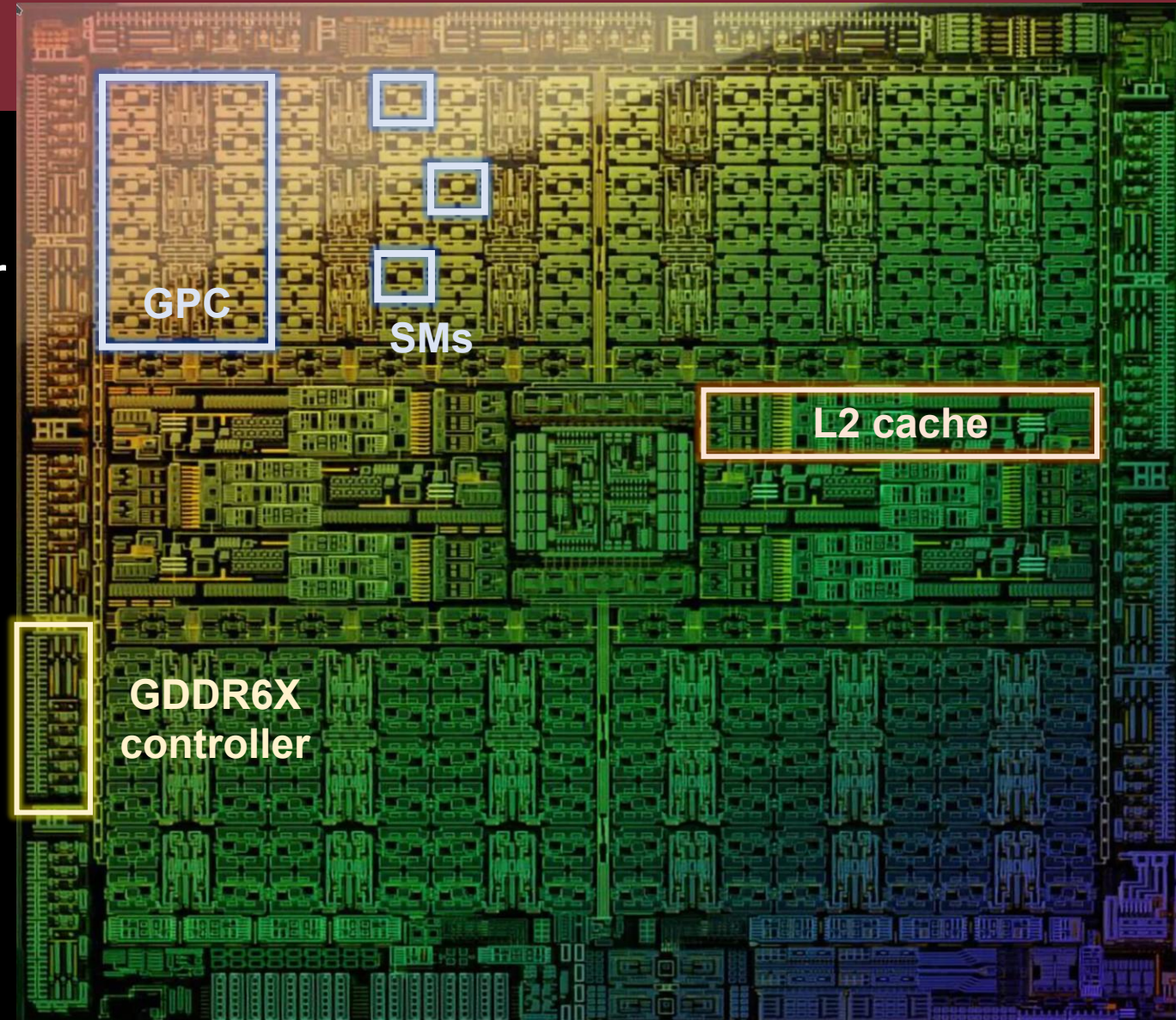


Intel Meteor Lake Die Shot



NVIDIA Ada Lovelace 102

- GPU: graphics processing unit
- GPC: graphics processing cluster
- SM: streaming multiprocessor
- GDDR: graphics double data rate
 - optimized for high bandwidth and parallel data transfers

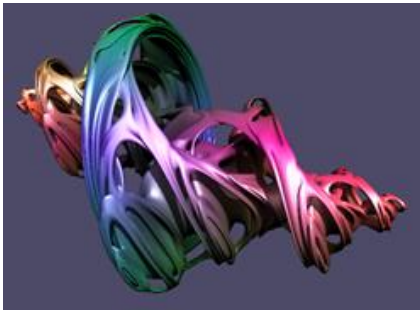


Flynn's Taxonomy

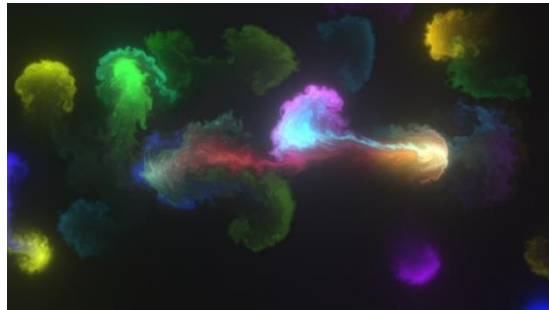
- **single instruction, single data (SISD)** – traditional process, no parallelism
- **single instruction, multiple data (SIMD)** – single instruction (or set of instructions) executed on different memory locations
 - array processors are sometimes called single instruction, multiple thread (SIMT)
- **multiple instruction, single data (MISD)**
 - used for redundancy (multiple programs compare results on the same data)
- **multiple instruction, multiple data (MIMD)** – parallel independent or unrelated processes
- **This course focuses on flexible programming integrating MIMD and SIMD concepts**

What's Next

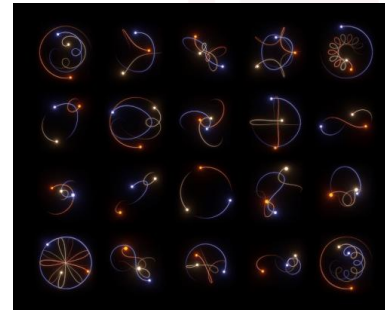
- Preparing for programming assignments
 - select the programming assignment



ray tracing



Navier-Stokes simulation



n-body simulation

- selecting your personal build platform
- High-level MIMD parallelism with threads
- Memory latency and bandwidth, modern processor architectures